Indian Institute of Technology, Bhilai

CSP203: Software Tools & Technologies Laboratory

Lab Activity - 15

Full Marks: 50 *Deadline: 11:59 PM, 25-10-2025*

Submission Instructions:

1) Answer all the questions.
2) You should prepare a zip file which will contain two reports, whose naming will be <Roll_No>_<Name>_Task_<Task No>.pdf. Each of these reports will contain the corresponding screenshots of the outputs and codes with question numbers.
3) Your zip file should be named as <Roll No>_<Name>_<Lab Activity No>.zip
4) Example: B24CS055_Shareen_15.zip.
5) Report Quality and Naming will contain 10 marks (5 Marks for each report).
6) **Plagiarism is strictly prohibited.**

Objectives:

1) Develop proficiency in core and advanced Git operations including initialization, configuration, staging, committing, branching, merging, rebasing, and tagging.
2) Apply version control concepts to manage and track file changes effectively in both local and remote repositories.
3) Simulate real-world collaborative workflows using branching strategies, forks, and remote synchronization through GitHub.
4) Handle and resolve common version control challenges such as merge conflicts, undoing changes, and maintaining a clean commit history.

5) Integrate comprehensive Git command usage with proper documentation and visualization of repository states for professional software development practices.

==========

Task1: [20 Marks]

Instructions: Perform the following Git operations **on your local machine** and document each step with appropriate commands and outputs (screenshots or command logs). Be precise and complete. Each question carries equal marks.

Q1. Repository Setup and Configuration [5 Marks]

Objective: Initialize a Git repository with personalized configuration.

Tasks:

- Create a new directory named vcs-assessment.
- Initialize a Git repository inside it.
- Configure your Git user identity (name, email) locally for this repo only.
- Set the default branch name to main.
- Create two files: info.txt and data.txt, add content In your local repository, add GitHub as a remote to each, and commit them.

Deliverables:

- Output of git config --list --show-origin
- git log showing the commit message and author info.

Q2. File Status and Differences [5 Marks]

Objective: Modify file contents and observe file states using Git tools.

Tasks:

- Modify data.txt by appending a new line.
- Use git status and git diff to display the working directory changes.

- Stage the changes using git add.
- Now run git diff --staged and commit the staged changes with an appropriate message.

Deliverables:

- Screenshots or command logs showing:
    - File status before and after staging.
    - Difference before and after staging.
    - Final commit message in log.

Q3. Undoing Operations [5 Marks]

Objective: Revert both staged and committed changes.

Tasks:

- Unstage any file using git reset (simulate by modifying and staging a file).
- Revert a file (e.g., info.txt) to its last committed state using git restore.
- Commit a change, then amend the commit to add a new line in the same file.

Deliverables:

- Commands used and outputs for:
    - Unstaging a file
    - Restoring a file
    - Amending the commit (show new commit hash via git log -1)

Q4. Remote Repository and Tagging [5 Marks]

Objective: Push local changes and manage tags on GitHub.

Tasks:

- Create a public GitHub repository named vcs-lab-test.
- Link your local repo (vcs-assessment) to this remote using git remote add.
- Push all commits to GitHub.

- Create one **annotated tag** (v1.0) and one **lightweight tag** (test-lite) locally.
- Push all tags to GitHub.

Deliverables:

- Output of:
  - git remote -v
  - git tag and git show v1.0
  - git push origin --tags
  - GitHub repository link

==========

Task2: [20 Marks]

Instructions:
Perform the following Git operations on your local machine and document each step with appropriate commands and outputs (screenshots or command logs). Be precise and complete. Each question carries equal marks.

Q1. Forking and Topic Branch Workflow [5 Marks]

Objective: Simulate a real-world collaborative development process using Git and GitHub.

Tasks:

- Fork the public GitHub repository: https://github.com/git-school/visualizing-git into your own GitHub account.
- Clone your forked repository to your local machine.
- Create a new branch named feature/footer-note from the default branch.
- Edit the index.html file and add a footer line at the bottom as an HTML comment, containing your full name and the current date.
- Stage and commit your changes with a meaningful message describing the new feature.
- Push the feature/footer-note branch to your forked GitHub repository (do **not** create a pull request).

Deliverables:

- Output of git branch showing the new branch.
- git log -1 showing the commit message, author, and timestamp.
- GitHub link to your forked repository showing the pushed branch.

Q2. Remote Branch Management and Cleanup [5 Marks]

Objective: Demonstrate remote branch operations and cleanup workflow.

Tasks:

- In any Git repository, create and switch to a new branch named hotfix/ui-glitch.
- Create or edit a file named ui.css and add a comment at the top indicating it's a hotfix.
- Stage and commit the changes with an appropriate hotfix-related commit message.
- Push the branch to your GitHub repository and set upstream.
- Now simulate branch cleanup:
    - Delete the hotfix/ui-glitch branch from the remote using Git.
    - Delete the local hotfix/ui-glitch branch as well.

Deliverables:

- Output of git branch -vv and git push logs.
- Output of git push origin --delete hotfix/ui-glitch.
- Final git branch output showing that the local branch is deleted.

Q3. Merge Conflict Creation and Manual Resolution [5 Marks]

Objective: Simulate a merge conflict and resolve it using manual editing.

Tasks:

- From your main branch, create and switch to a new branch called branch-A.
- In branch-A, create or edit a file named style.css and add the line: Color: red;. Stage and commit the change.
- Switch back to the main branch.

- Modify the same file style.css and add the line: Color: blue;. Stage and commit this change.
- Attempt to merge branch-A into main. A merge conflict should occur in style.css.
- Open the conflicted file and resolve the conflict manually by preserving both styles with clear comment sections.
- Stage and commit the resolved file.

Deliverables:

- Screenshot or output of the merge conflict message.
- Original conflict markers in style.css and the resolved version.
- Output of git log -1 showing the conflict resolution commit.

Q4. Rebasing and Clean Merge Integration [5 Marks]

Objective: Perform a rebase operation and integrate the rebased branch using a fast-forward merge.

Tasks:

- Create a new branch named feature/rebase-demo from the current main branch.
- In feature/rebase-demo, create a file called rebase.js and add the line: console.log('Rebase demo');.
- Stage and commit this file.
- Switch to main and add a new file fix.css with a CSS comment or bugfix. Commit the change.
- Switch back to feature/rebase-demo and rebase it onto the latest main.
- After a successful rebase, switch to main and perform a fast-forward merge of the feature/rebase-demo branch.
- Delete the feature/rebase-demo branch after merge.

Deliverables:

- Output of git rebase showing success or conflict (if any).
- Output of git log --oneline --graph --all to demonstrate linear history.
- Confirmation of branch deletion using git branch -d feature/rebase-demo.